# Practical Work 3: Document Databases and Hierarchical Data Models

**Variation No. 19: The Financial Fraud Analyst**

**Didem Nur Mutlu**

**221ADB286**

**Video Link: https://youtu.be/8uu4pvwC4nE**

# The Scenario: "FinSecure Banking"

Welcome to FinSecure Banking. You have just been hired as a Junior Data Engineer in the Fraud Detection Unit. The bank is facing a spike in suspicious activities. Historically, transaction logs, account details, and merchant blacklists have been stored in disjointed CSV and JSON files. This fragmentation is making it impossible for the "Risk Management Team" to calculate risk exposure, flag high-value suspicious transfers, or analyze spending patterns by merchant category.

Your Task: Your Chief Risk Officer (CRO) has tasked you with migrating this legacy data into a MongoDB Document Database. You must move the data from flat files into a hierarchical format, clean the data types, and perform a series of advanced analytical queries.

# Task 1: Installation process

1. A step-by-step description of how you installed MongoDB.

I navigated to the official MongoDB download page at https://www.mongodb.com/try/download/community, Selected the appropriate version for my Windows operating system then downloaded the MSI installer package, Ran the installer with administrator privileges, During installation, I selected "Complete" installation type, In the "Service Configuration" screen, I chose to install MongoDB as a Windows Service with "Run service as Network Service user" option, I then ensured that "Install MongoDB Compass" option was checked and thus completed the installation process

2. MongoDB Compass Installation:

MongoDB Compass was automatically installed as part of the Community Server installation, Alternatively, it can be downloaded separately from,
https://www.mongodb.com/try/download/compass

3. Environment Configuration:

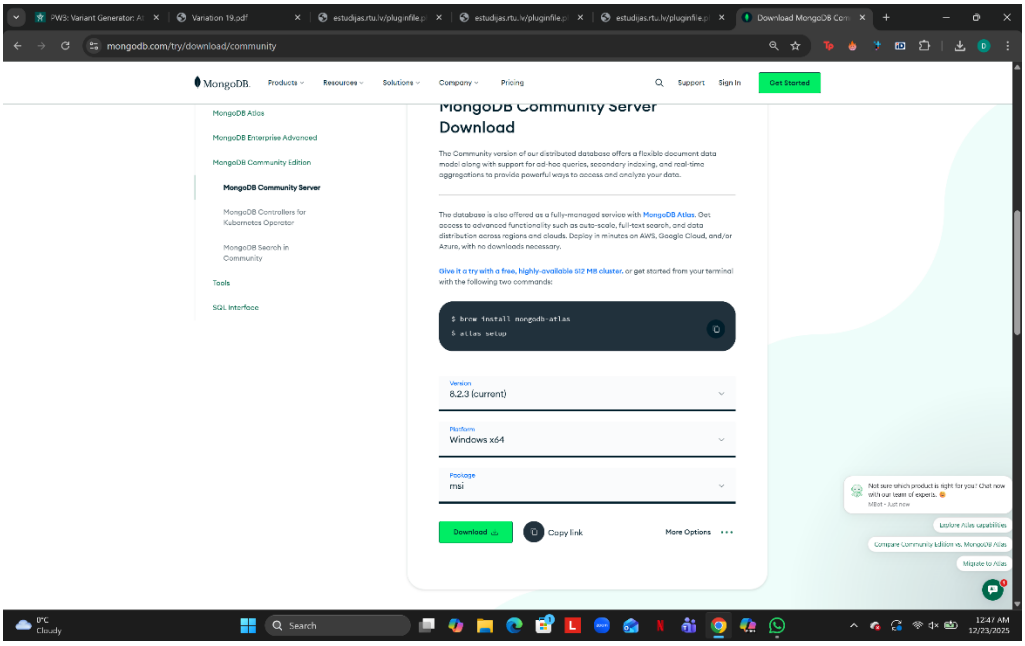
   a. Added MongoDB bin directory to Windows PATH environment variable: C:\Program Files\MongoDB\Server\8.2\bin
   b. This allows running mongod and mongosh commands from any terminal location

4. Verification:

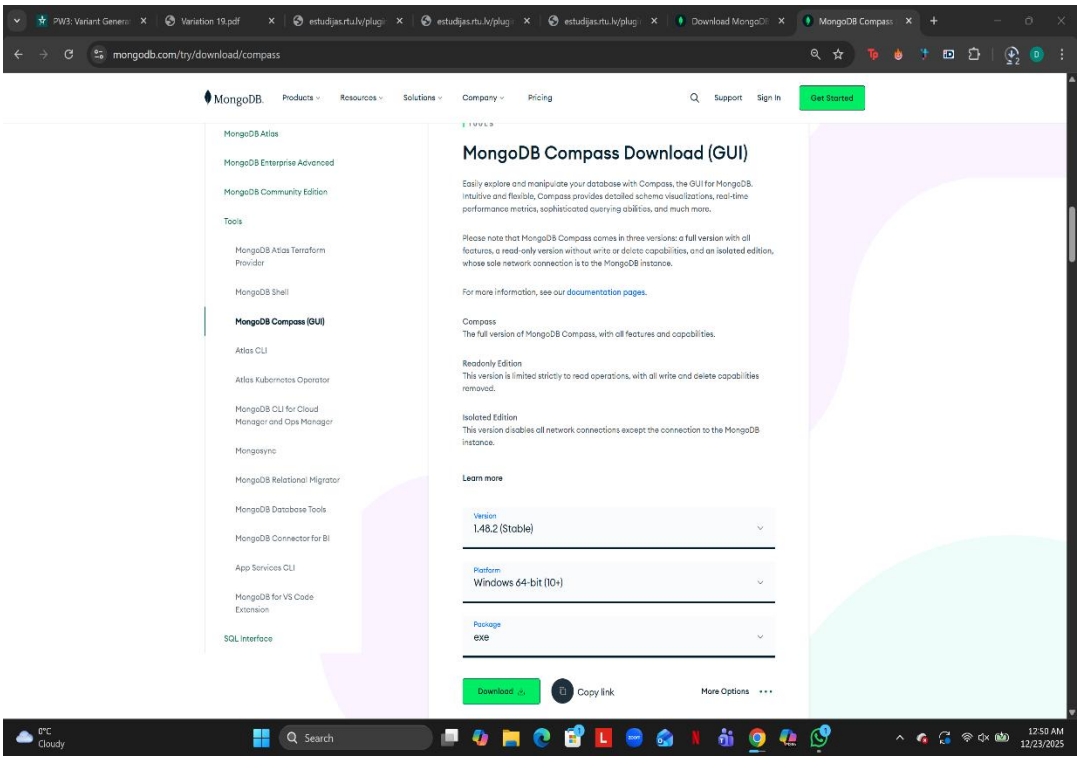
   a. Opened Windows PowerShell
   b. Ran mongod --version to verify MongoDB server installation
   c. Ran mongosh --version to verify MongoDB shell installation
   d. Started MongoDB service using net start MongoDB (as administrator)
   e. Connected to MongoDB shell using mongosh command
   f. Successfully established connection to mongodb://localhost:27017

2. Screenshots of:

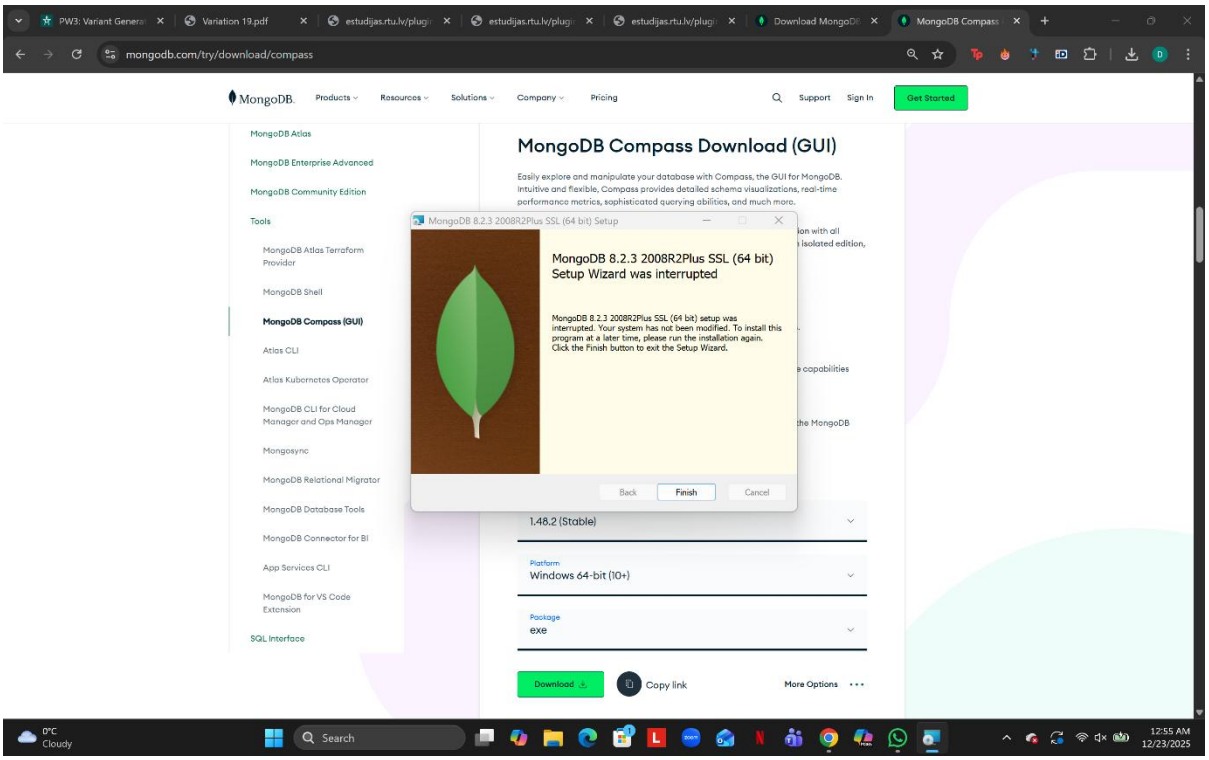
3. MongoDB Community Server Download page with your specific download Version, Platform and Package.



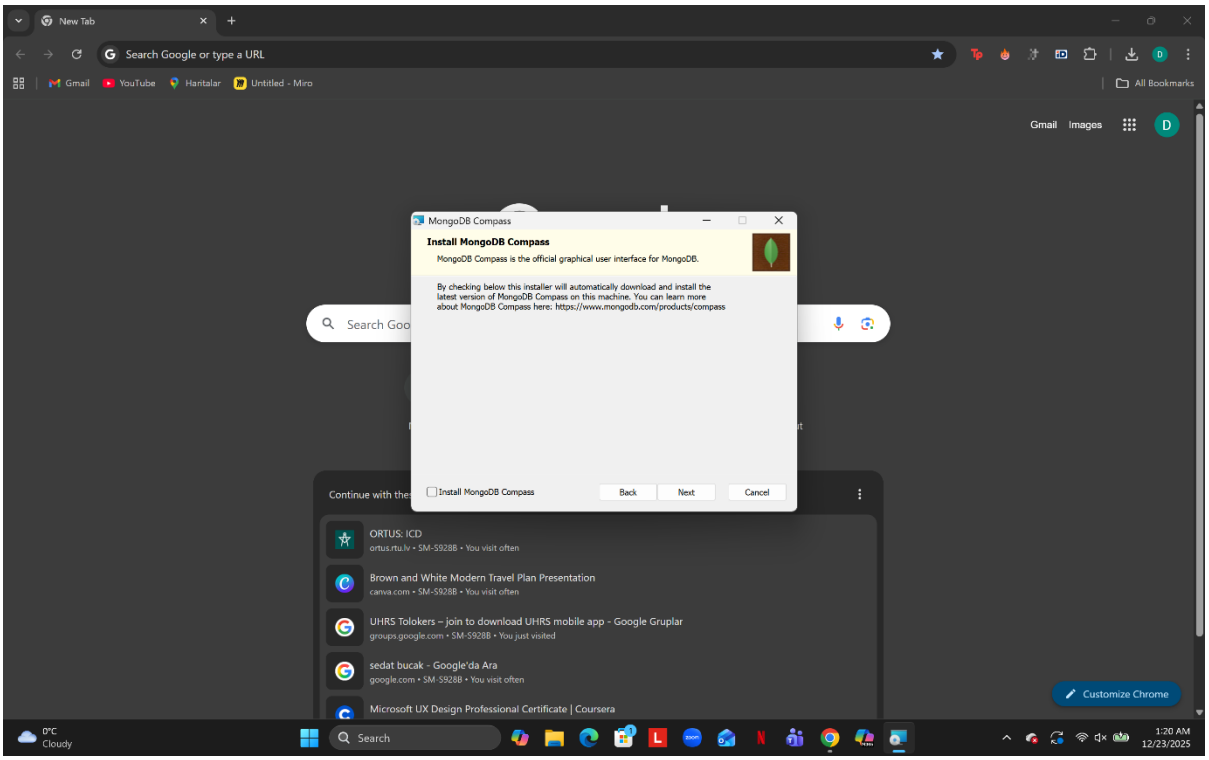

4. MongoDB Compass Download page with your specific download Version, Platform and Package.



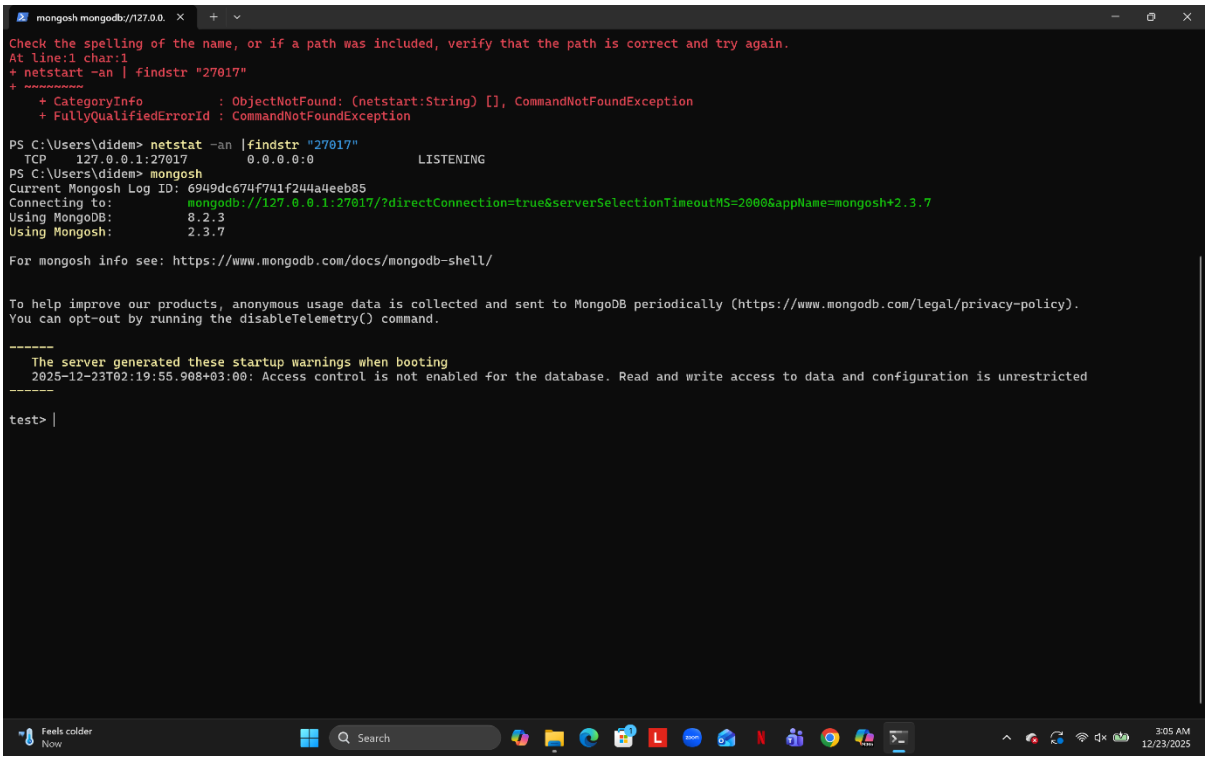
5. The "Component Selection" screen during installation.




6. The "Installation Complete" screen.



7. A screenshot of the terminal showing a successful connection to mongosh



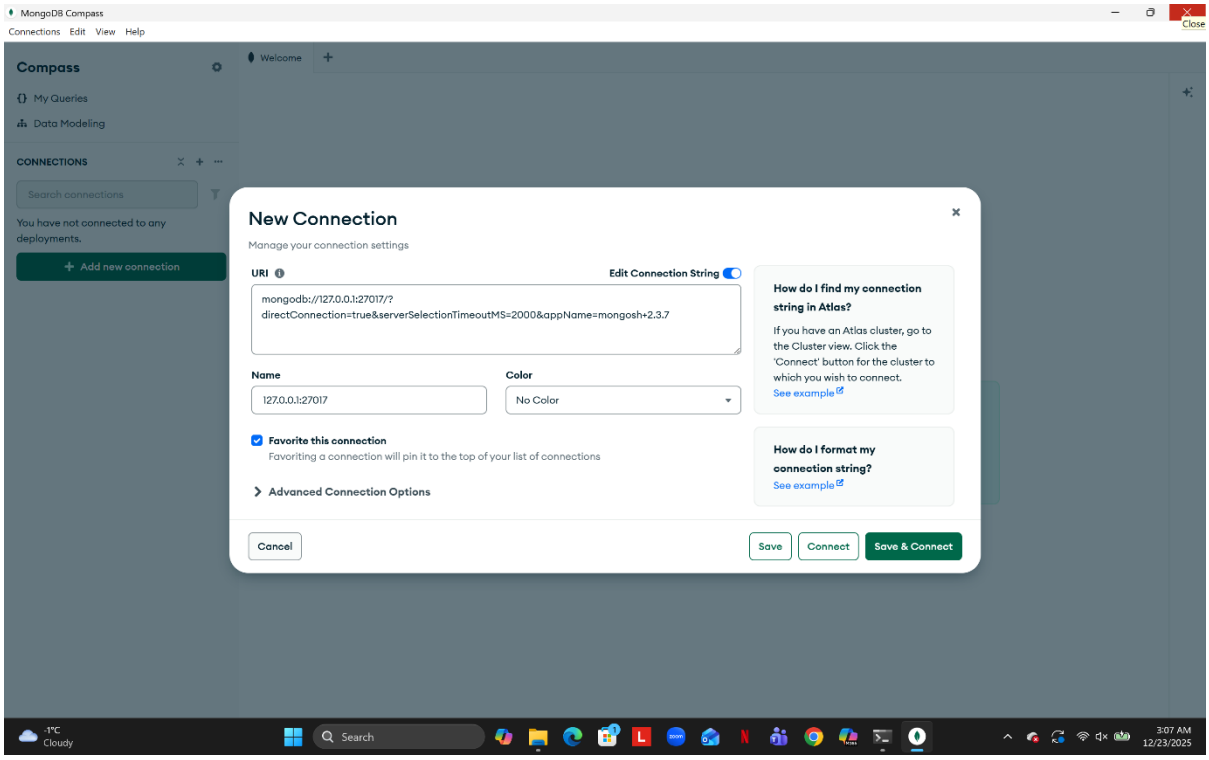
# Task 2: Data import and verification

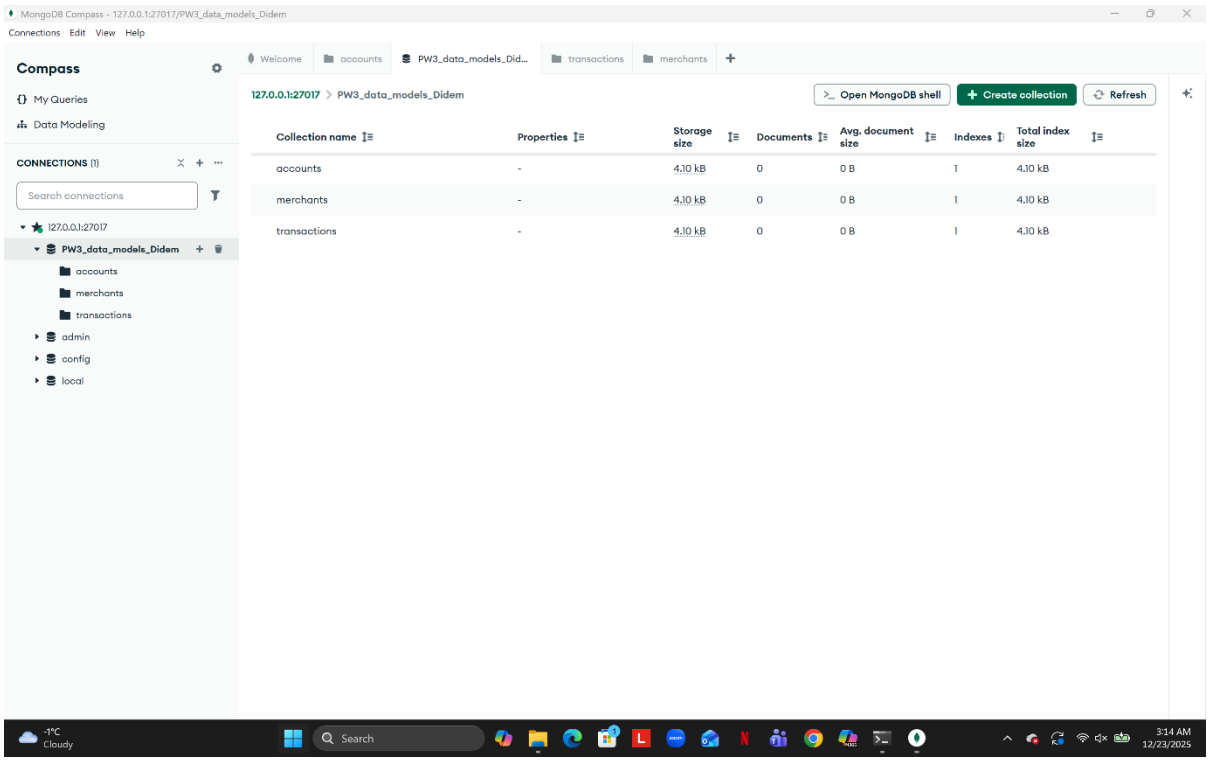
1. Description:

After setting up MongoDB, I created a database named "financeDB" for the fraud detection system. Within it, I established three key collections: transactions, accounts, and merchants. Using MongoDB Compass, I imported the corresponding JSON files into each collection, which was a straightforward visual process.

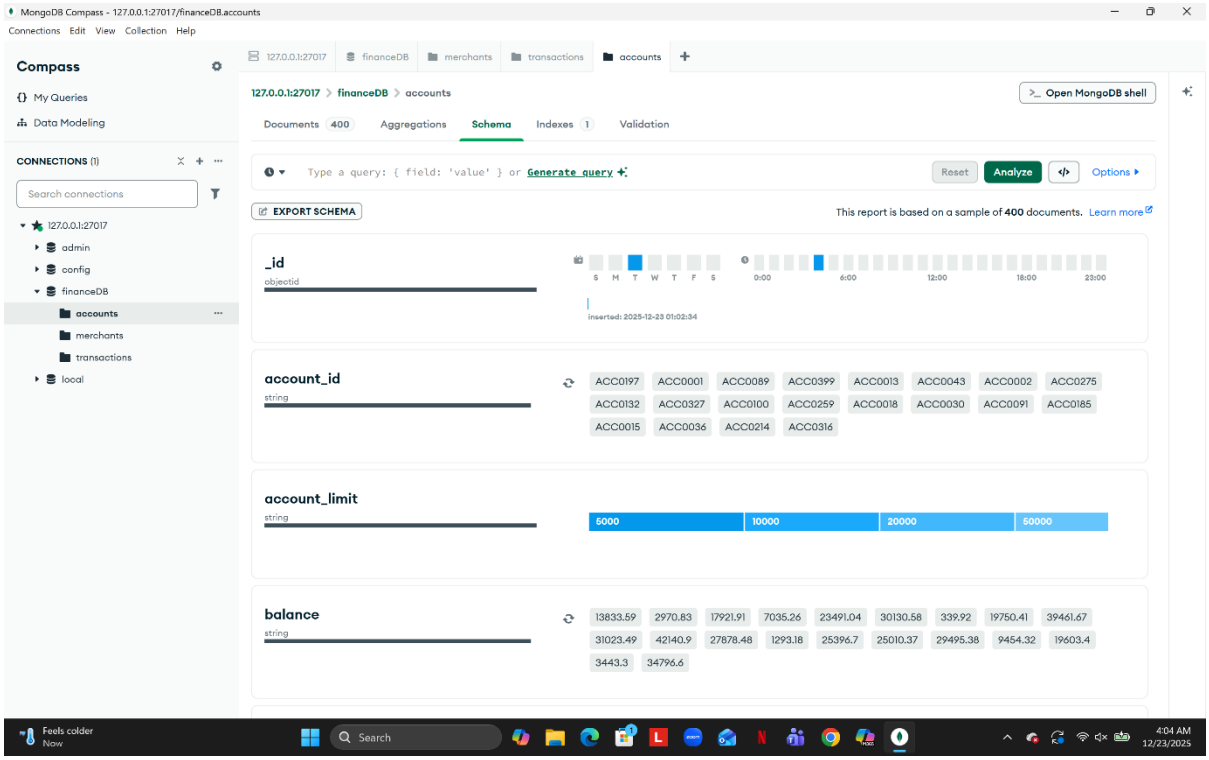
However, a significant issue emerged immediately after the import: all the data fields were stored as strings. This meant critical values like transaction amounts, timestamps, and balances could not be used for calculations or date comparisons, requiring a data type conversion as the next essential step..

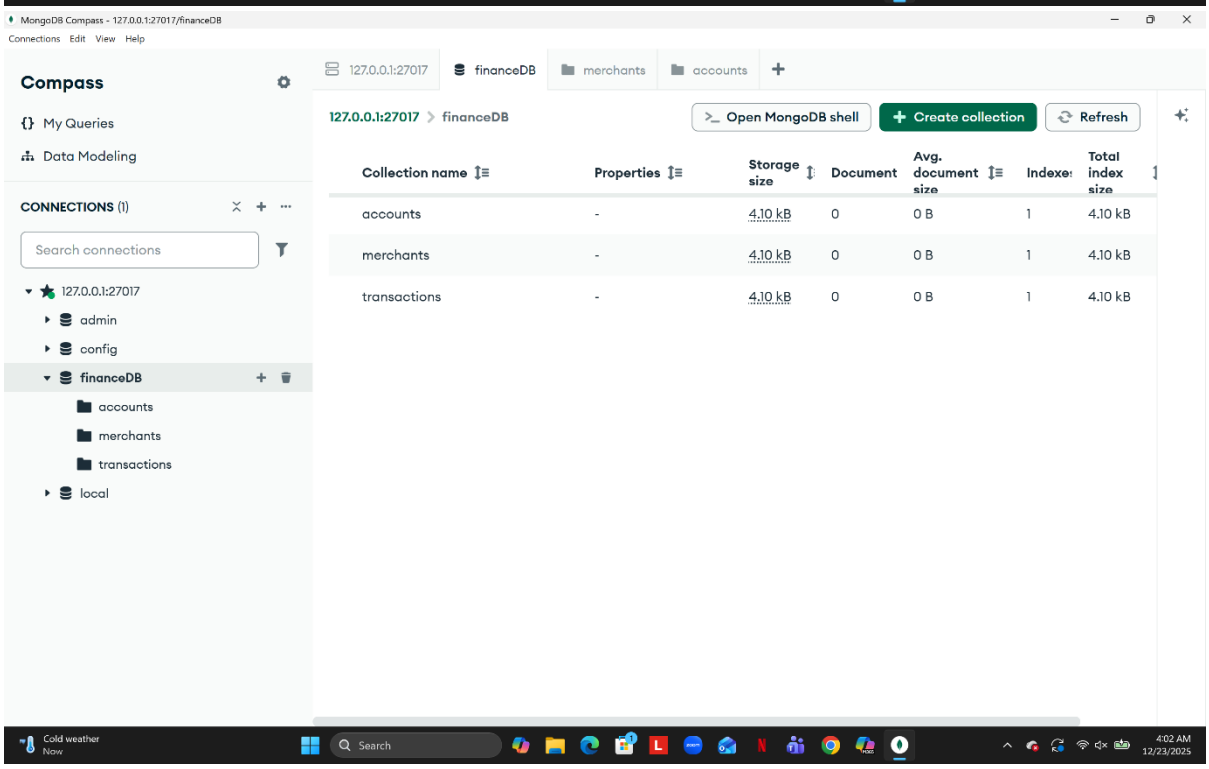
2. Screenshots of:
   MongoDB Compass of the created database.

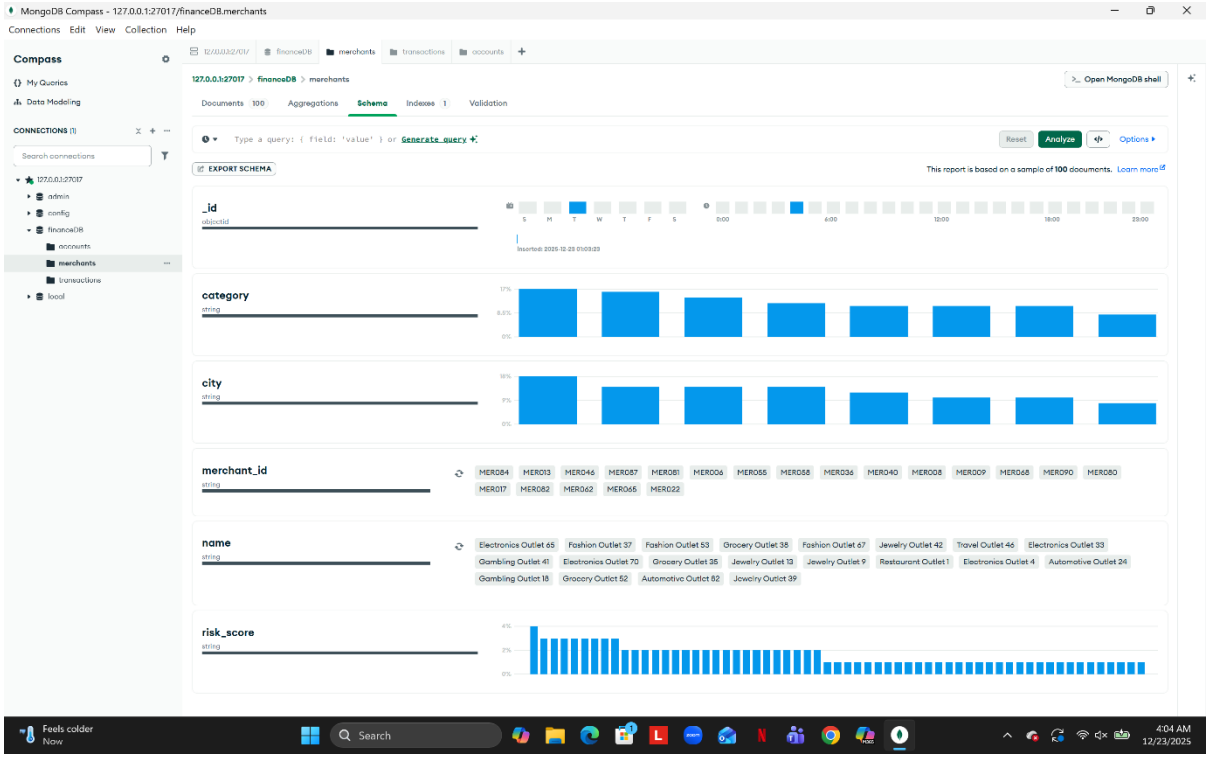

MongoDB Compass
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS
Search connections
You have not connected to any deployments.
Add new connection
Welcome
New Connection
Manage your connection settings
URI
Edit Connection String
mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.3.7
Name
127.0.0.1:27017
Color
No Color
Favorite this connection
Favoriting a connection will pin it to the top of your list of connections
Advanced Connection Options
How do I find my connection string in Atlas?
If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.
See example
How do I format my connection string?
See example
Cancel
Save
Connect
Save & Connect

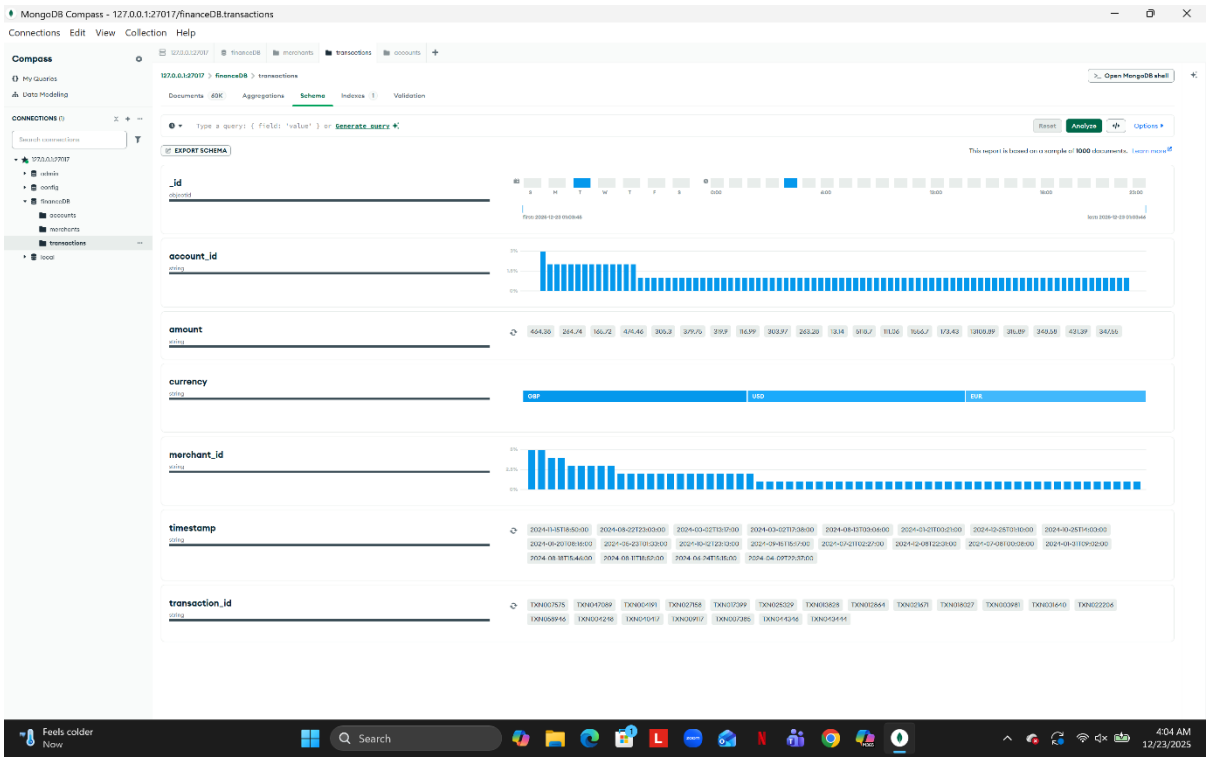

MongoDB Compass - 127.0.0.1:27017/PW3_data_models_Didem
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
127.0.0.1:27017
PW3_data_models_Didem
accounts
merchants
transactions
admin
config
local
Welcome
accounts
PW3_data_models_Did...
transactions
merchants
127.0.0.1:27017 > PW3_data_models_Didem
Open MongoDB shell
Create collection
Refresh
Collection name
Properties
Storage size
Documents
Avg. document size
Indexes
Total index size
accounts - 4.10 kB 0 0 B 1 4.10 kB
merchants - 4.10 kB 0 0 B 1 4.10 kB
transactions - 4.10 kB 0 0 B 1 4.10 kB

3. MongoDB Compass for each of the 3 collections.




MongoDB Compass - 127.0.0.1:27017/financeDB.merchants
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
127.0.0.1:27017
admin
config
financeDB
accounts
merchants
transactions
local
127.0.0.1:27017 > financeDB > merchants
Open MongoDB shell
Documents 100
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Reset
Analyze
Options
EXPORT SCHEMA
This report is based on a sample of 100 documents. Learn more
_id
objectid
category
string
city
string
merchant_id
string
MER084 MER013 MER046 MER087 MER081 MER006 MER055 MER058 MER036 MER040 MER008 MER009 MER068 MER090 MER080
MER017 MER082 MER062 MER065 MER022
name
string
Electronics Outlet 65 Fashion Outlet 37 Fashion Outlet 53 Grocery Outlet 38 Fashion Outlet 67 Jewelry Outlet 42 Travel Outlet 46 Electronics Outlet 33
Gambling Outlet 41 Electronics Outlet 70 Grocery Outlet 35 Jewelry Outlet 13 Jewelry Outlet 9 Restaurant Outlet 1 Electronics Outlet 4 Automotive Outlet 24
Gambling Outlet 18 Grocery Outlet 52 Automotive Outlet 82 Jewelry Outlet 39
risk_score
string
Feels colder
Now
Search
4:04 AM
12/23/2025


MongoDB Compass - 127.0.0.1:27017/financeDB.transactions
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
127.0.0.1:27017
admin
config
financeDB
accounts
merchants
transactions
local
127.0.0.1:27017 > financeDB > transactions
Open MongoDB shell
Documents 60K
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Reset
Analyze
Options
EXPORT SCHEMA
This report is based on a sample of 1000 documents. Learn more
_id
objectid
account_id
string
amount
string
currency
string
GBP
USD
EUR
merchant_id
string
timestamp
string
transaction_id
string
Feels colder
Now
Search
4:04 AM
12/23/2025

# Task 3: Data Type Correction

1. Process: In detail explain the script used to fix the types.

After importing json files, the data was stored as strings was going to cause major problems when trying to do any kind of analysis. Financial calculations need actual numbers, and time-based queries need proper date objects. So, I wrote three different update scripts to fix the data types in each collection.

For the merchants collection, the main issue was the risk_score field. This field is supposed to be a number that represents how risky a particular merchant is, but it was imported as a string. I wrote a script using MongoDB's updateMany function to find all documents where risk_score was a string and convert it to an Int32 (a 32-bit integer). The way this works is an aggregation pipeline inside the update operation, which lets you do transformations on the data as you're updating it. The filter part of my script looks for documents where the risk_score field has a type of "string", and then the update part uses the $toInt operator to convert that string value into an actual integer. This way, MongoDB can properly handle the risk scores as numbers in calculations later.

The accounts collection had two fields that needed fixing: account_limit and balance. Both of these represent money amounts, so they need to be stored as Double type (which is a floating-point number) to handle decimal places correctly. My script for this collection was a bit more complex because I needed to check if either field was a string, so I used the $or operator in the filter. Then in the update portion, I converted both fields at the same time using $toDouble. This ensures that when we do financial calculations later, like checking if someone is over their limit or calculating total balances, the math will work correctly and give us precise results with decimal places.

The transactions collection was the most important one to fix because it's where all the actual transaction data is stored. This collection had two fields with the wrong types: amount and timestamp. The amount field needs to be a Double so we can do all sorts of calculations like summing up totals, finding averages, and comparing transaction sizes. The timestamp field needs to be a proper Date object so we can do time-based queries like finding transactions from a specific day or sorting by when they occurred. The script checked for both fields being strings using the $or operator again, and then converted amount to Double with $toDouble and timestamp to Date with $toDate.

Screenshots of:

a. MongoDB Compass showing that the data types have been successfully changed to their correct formats (e.g. form String to Int, Dates etc.) after running your script for each of the collections (even if a collection does not change for your specific task requirements).

MongoDB Compass - 127.0.0.1:27017/Shell

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

127.0.0.1:27017

admin

config

financeDB

accounts

merchants

transactions

local

mongosh: 127.0.0.1:27017

>_MONGOSH

```
> use ('financeDB');
< switched to db financeDB
>
  /* ==========================================================================
     STUDENT HEADER
     Course: Data models in database systems
     Practical Work 3: Document databases and hierarchical data model

     Student Name, Surname:  [ENTER YOUR NAME HERE]
     Student ID:             [ENTER YOUR ID HERE]
     Variation #:            19
  ========================================================================== */

  // --- SECTION 2: DATABASE SETUP ---
  // Switch to the financeDB database - this ensures all operations run on the correct database
  use('financeDB');
  print(">>> Database 'financeDB' selected.");


  /* ==========================================================================
     SECTION 3: TASK 3 - DATA TYPE CORRECTION
     Purpose: Convert string data types to their correct formats for analytical processing
     Collections affected: merchants, accounts, transactions
  ========================================================================== */

  print("\n>>> Running Task 3: Type Correction...");

< >>> Database 'financeDB' selected.
<
  >>> Running Task 3: Type Correction...
financeDB>
```

-1°C Partly sunny

Search

3:33 PM 12/23/2025

MongoDB Compass - 127.0.0.1:27017/Shell

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

127.0.0.1:27017

admin

config

financeDB

accounts

merchants

transactions

local

mongosh: 127.0.0.1:27017

>_MONGOSH

```
>
  /* ==========================================================================
     STUDENT HEADER
     Course: Data models in database systems
     Practical Work 3: Document databases and hierarchical data model

     Student Name, Surname:  [ENTER YOUR NAME HERE]
     Student ID:             [ENTER YOUR ID HERE]
     Variation #:            19
  ========================================================================== */

  // --- SECTION 2: DATABASE SETUP ---
  // Switch to the financeDB database - this ensures all operations run on the correct database
  use('financeDB');
  print(">>> Database 'financeDB' selected.");


  /* ==========================================================================
     SECTION 3: TASK 3 - DATA TYPE CORRECTION
     Purpose: Convert string data types to their correct formats for analytical processing
     Collections affected: merchants, accounts, transactions
  ========================================================================== */

  print("\n>>> Running Task 3: Type Correction...");

< >>> Database 'financeDB' selected.
<
  >>> Running Task 3: Type Correction...
>
  // 3.1 Fix Collection 1: merchants
  // Purpose: Convert risk_score from String to Int32 for numerical calculations
  // Process: Find all documents where risk_score is a string, then convert to integer
  print("\n[3.1] Fixing merchants collection: risk_score String → Int32");
  db.merchants.updateMany(
    // Filter: Select documents where risk_score field is of type string
    { risk_score: { $type: "string" } },
    // Update: Use aggregation pipeline to convert string to integer
    [{
      $set: {
        risk_score: { $toInt: "$risk_score" }  // Convert current string value to Int32
      }
    }]
  );
  print(" merchants.risk_score converted to Int32");


<
  [3.1] Fixing merchants collection: risk_score String → Int32
<  merchants.risk_score converted to Int32
financeDB>
```

-1°C Partly sunny

Search

3:35 PM 12/23/2025

MongoDB Compass - 127.0.0.1:27017/Shell

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

127.0.0.1:27017

admin

config

financeDB

accounts

merchants

transactions

local

mongosh: 127.0.0.1:27017

>_MONGOSH

```
< >>> Database 'financeDB' selected.
<
  >>> Running Task 3: Type Correction...
>
  // 3.1 Fix Collection 1: merchants
  // Purpose: Convert risk_score from String to Int32 for numerical calculations
  // Process: Find all documents where risk_score is a string, then convert to integer
  print("\n[3.1] Fixing merchants collection: risk_score String > Int32");
  db.merchants.updateMany(
    // Filter: Select documents where risk_score field is of type string
    { risk_score: { $type: "string" } },
    // Update: Use aggregation pipeline to convert string to integer
    [{
      $set: {
        risk_score: { $toInt: "$risk_score" }  // Convert current string value to Int32
      }
    }]
  );
  print(" merchants.risk_score converted to Int32");

<
  [3.1] Fixing merchants collection: risk_score String → Int32
<  merchants.risk_score converted to Int32
> // 3.2 Fix Collection 2: accounts
  // Purpose: Convert account_limit and balance from String to Double for financial calculations
  // Process: Find documents with string values in either field and convert both to doubles
  print("\n[3.2] Fixing accounts collection: account_limit & balance String → Double");
  db.accounts.updateMany(
    // Filter: Select documents where either field is a string
    {
      $or: [
        { account_limit: { $type: "string" } },
        { balance: { $type: "string" } }
      ]
    },
    // Update: Convert both fields to Double simultaneously
    [{
      $set: {
        account_limit: { $toDouble: "$account_limit" },  // Convert to Double for precise decimal values
        balance: { $toDouble: "$balance" }               // Convert to Double for precise decimal values
      }
    }]
  );
  print(" accounts.account_limit and accounts.balance converted to Double");
<
  [3.2] Fixing accounts collection: account_limit & balance String > Double
<  accounts.account_limit and accounts.balance converted to Double
financeDB>
```

-1°C Partly sunny

Search

3:36 PM 12/23/2025

MongoDB Compass - 127.0.0.1:27017/Shell

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

127.0.0.1:27017

admin

config

financeDB

accounts

merchants

transactions

local

mongosh: 127.0.0.1:27017

>_MONGOSH

```
  db.accounts.updateMany(
    // Filter: Select documents where either field is a string
    {
      $or: [
        { account_limit: { $type: "string" } },
        { balance: { $type: "string" } }
      ]
    },
    // Update: Convert both fields to Double simultaneously
    [{
      $set: {
        account_limit: { $toDouble: "$account_limit" },  // Convert to Double for precise decimal values
        balance: { $toDouble: "$balance" }               // Convert to Double for precise decimal values
      }
    }]
  );
  print(" accounts.account_limit and accounts.balance converted to Double");
<
  [3.2] Fixing accounts collection: account_limit & balance String → Double
<  accounts.account_limit and accounts.balance converted to Double
> // 3.3 Fix Collection 3: transactions
  // Purpose: Convert amount to Double and timestamp to Date for proper analytics and time-series operations
  // Process: Find documents with string values and convert to appropriate types
  print("\n[3.3] Fixing transactions collection: amount String → Double, timestamp String → Date");
  db.transactions.updateMany(
    // Filter: Select documents where either field is a string
    {
      $or: [
        { amount: { $type: "string" } },
        { timestamp: { $type: "string" } }
      ]
    },
    // Update: Convert amount to Double and timestamp to Date
    [{
      $set: {
        amount: { $toDouble: "$amount" },        // Convert to Double for financial calculations
        timestamp: { $toDate: "$timestamp" }     // Convert to Date for time-based queries
      }
    }]
  );
  print("transactions.amount converted to Double, transactions.timestamp converted to Date");

  print("\nTask 3 Complete: All data types have been corrected.");
<
  [3.3] Fixing transactions collection: amount String > Double, timestamp String > Date
< transactions.amount converted to Double, transactions.timestamp converted to Date
<
  Task 3 Complete: All data types have been corrected.
financeDB>
```

2°C Cloudy

Search

4:08 PM 12/23/2025

MongoDB Compass - 127.0.0.1:27017/Shell

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

127.0.0.1:27017

admin

config

financeDB

accounts

merchants

transactions

local

mongosh: 127.0.0.1:27017

```
>_MONGOSH
db.accounts.updateMany(
  // Filter: Select documents where either field is a string
  {
    $or: [
      { account_limit: { $type: "string" } },
      { balance: { $type: "string" } }
    ]
  },
  // Update: Convert both fields to Double simultaneously
  [{
    $set: {
      account_limit: { $toDouble: "$account_limit" },  // Convert to Double for precise decimal values
      balance: { $toDouble: "$balance" }               // Convert to Double for precise decimal values
    }
  }]
);
print(" accounts.account_limit and accounts.balance converted to Double");
{
  [3.2] Fixing accounts collection: account_limit & balance String → Double
{  accounts.account_limit and accounts.balance converted to Double
> // 3.3 Fix Collection 3: transactions
  // Purpose: Convert amount to Double and timestamp to Date for proper analytics and time-series operations
  // Process: Find documents with string values and convert to appropriate types
  print("\n[3.3] Fixing transactions collection: amount String → Double, timestamp String → Date");
  db.transactions.updateMany(
    // Filter: Select documents where either field is a string
    {
      $or: [
        { amount: { $type: "string" } },
        { timestamp: { $type: "string" } }
      ]
    },
    // Update: Convert amount to Double and timestamp to Date
    [{
      $set: {
        amount: { $toDouble: "$amount" },        // Convert to Double for financial calculations
        timestamp: { $toDate: "$timestamp" }     // Convert to Date for time-based queries
      }
    }]
  );
  print("transactions.amount converted to Double, transactions.timestamp converted to Date");

  print("\nTask 3 Complete: All data types have been corrected.");
{
  [3.3] Fixing transactions collection: amount String → Double, timestamp String → Date
{ transactions.amount converted to Double, transactions.timestamp converted to Date
{
  Task 3 Complete: All data types have been corrected.
financeDB>
```

2°C Cloudy

Search

4:08 PM 12/23/2025

# Task 4: Denormalization

1. Script title: Transaction-Merchant Integration with Risk Exposure Calculation

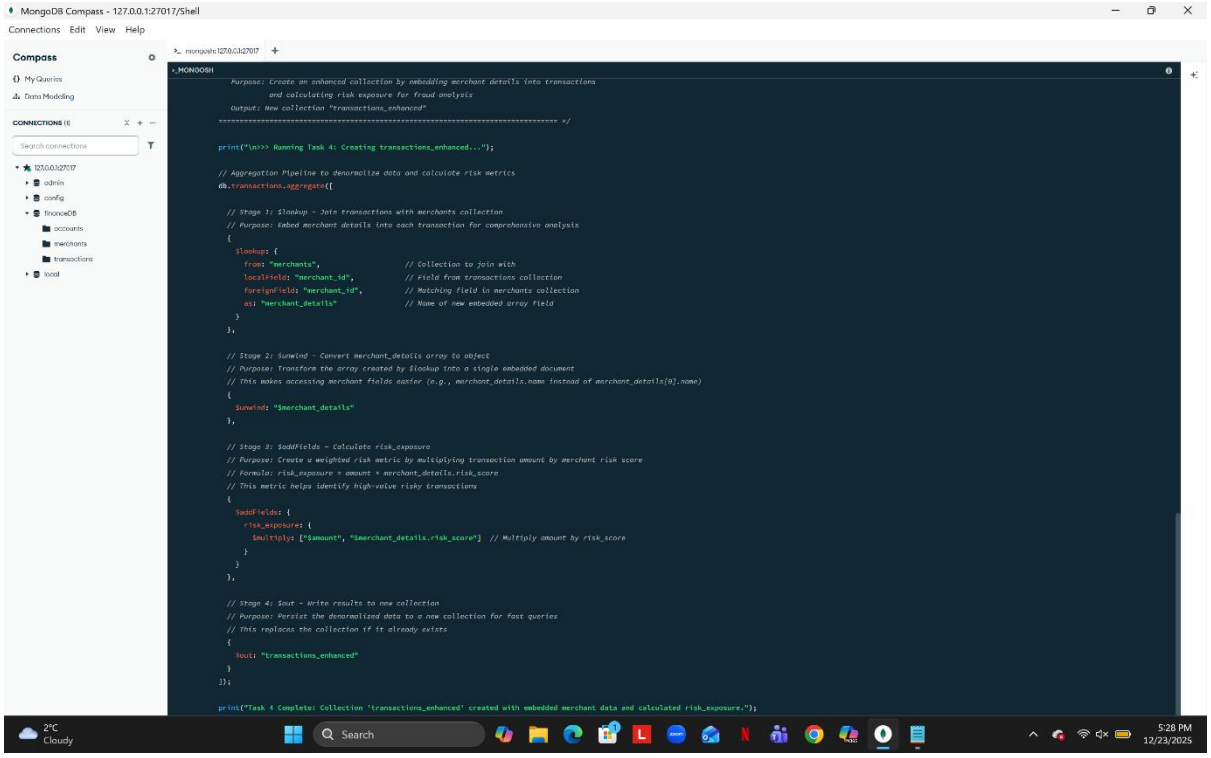
   Purpose (Semantic Explanation)
   This script creates a denormalized collection that embeds merchant details into each transaction and calculates a risk exposure metric for fraud analysis.

   Process (Technical Explanation)
   The script uses an aggregation pipeline with four stages:
   a. First, the $lookup stage joins the transactions collection with the merchants collection by matching merchant_id fields. It creates a new array field called merchant_details containing the matched merchant document.

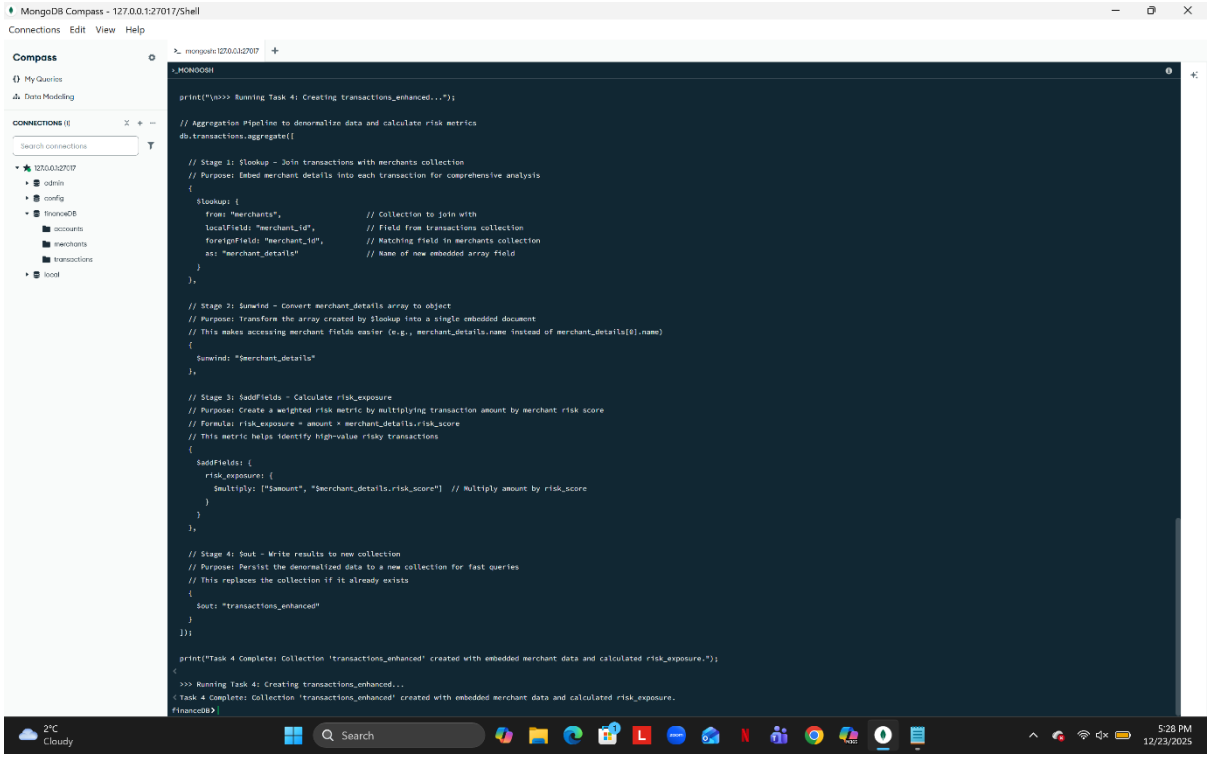
   b. Second, the $unwind stage converts the merchant_details array into a single object, making it easier to access merchant fields in subsequent stages.

   c. Third, the $addFields stage calculates a new field called risk_exposure by multiplying the transaction amount by the merchant's risk_score using the $multiply operator.
   d. Fourth, the $out stage writes all the transformed documents into a new permanent collection named transactions_enhanced.

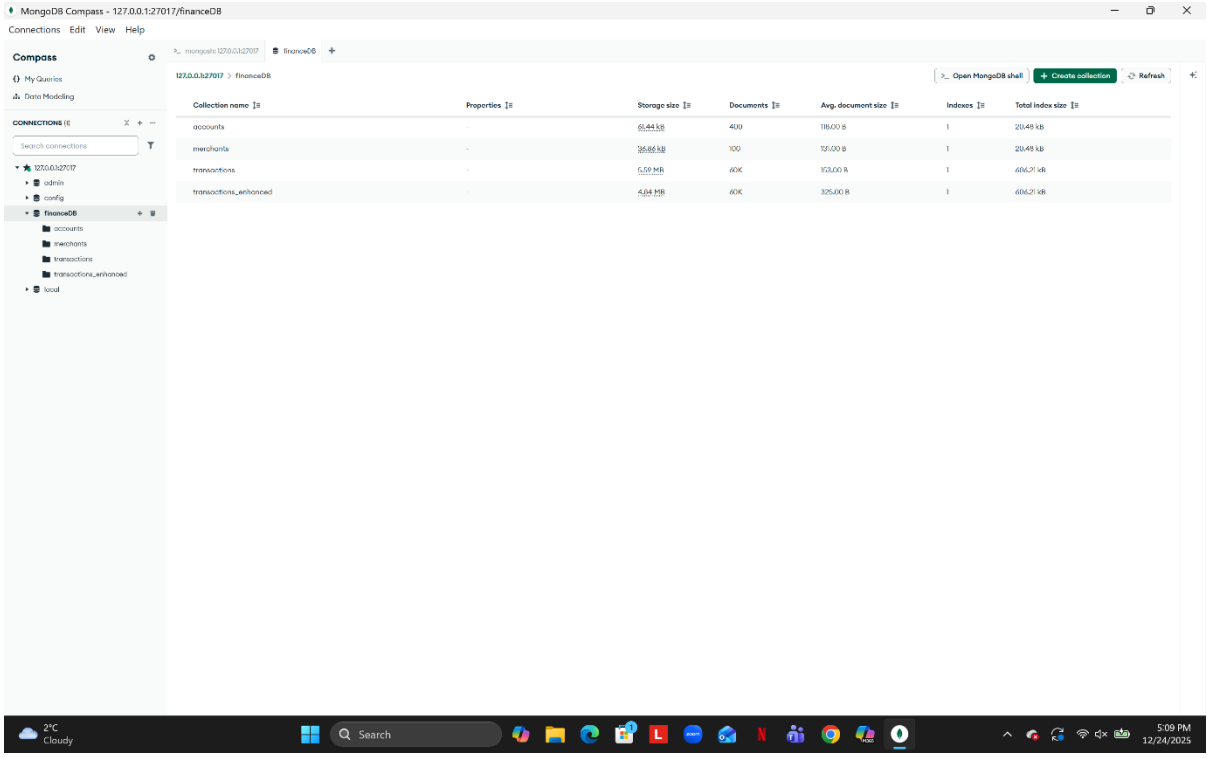
2. Screenshots of:
   a. The script




b. The output of the script



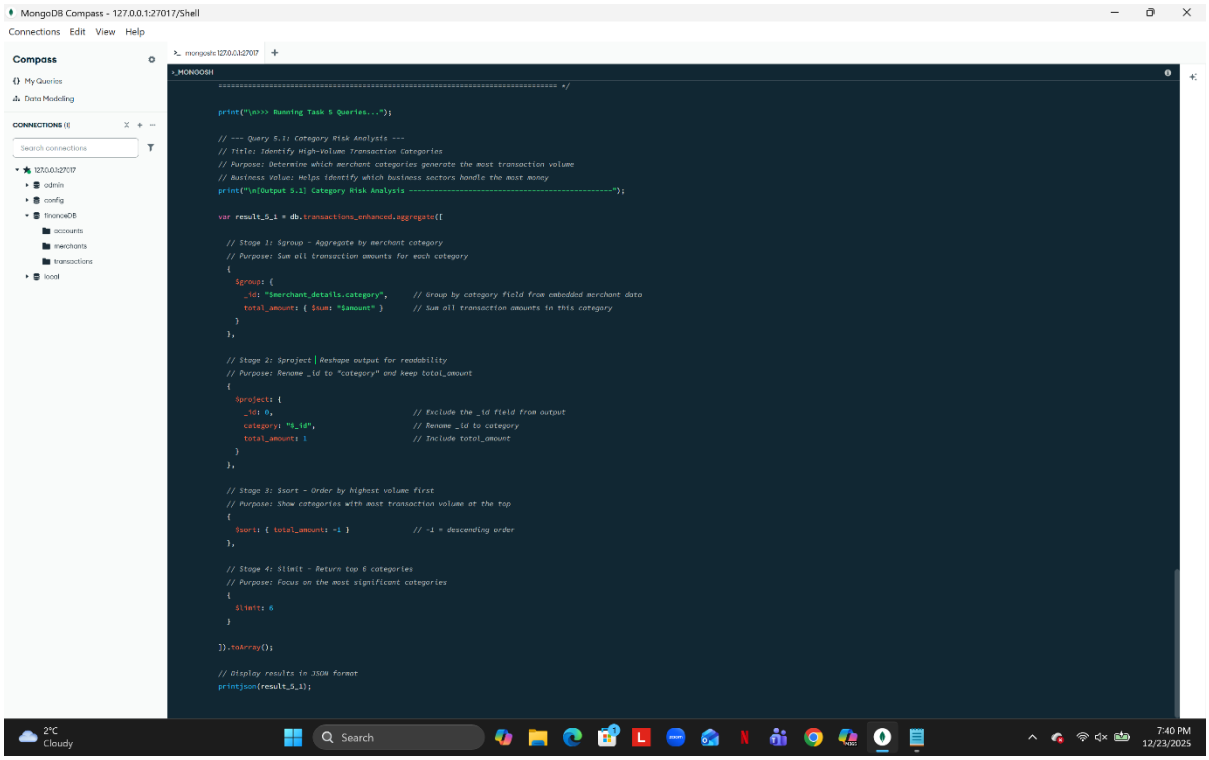

The new denormalized collection in MongoDB Compass:

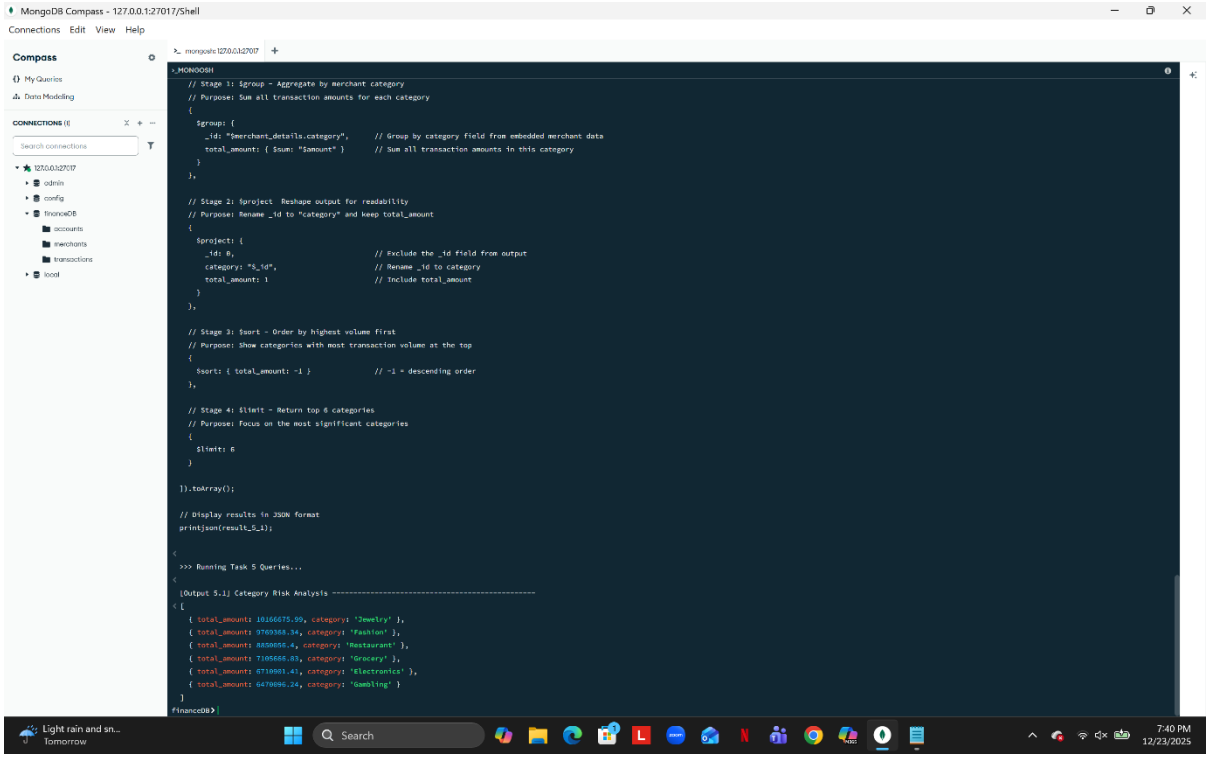

MongoDB Compass - 127.0.0.1:27017/financeDB
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
127.0.0.1:27017
admin
config
financeDB
accounts
merchants
transactions
transactions_enhanced
local
mongosh: 127.0.0.1:27017
financeDB
127.0.0.1:27017 > financeDB
Open MongoDB shell
Create collection
Refresh
Collection name
Properties
Storage size
Documents
Avg. document size
Indexes
Total index size
accounts - 61.44 kB 400 118.00 B 1 20.48 kB
merchants - 36.86 kB 100 131.00 B 1 20.48 kB
transactions - 5.59 MB 60K 193.00 B 1 606.21 kB
transactions_enhanced - 4.84 MB 60K 325.00 B 1 606.21 kB
2°C
Cloudy
Search
5:09 PM
12/24/2025

# Task 5: Advanced Queries

## Task 5.1

1. Query Title: High-Volume Transaction Categories Identification
2. Purpose (Semantic Explanation): This query identifies which merchant categories generate the most transaction volume by calculating total amounts per category.
3. Process (Technical Explanation):
    a. The query uses a four-stage aggregation pipeline. First, the $group stage groups all transactions by merchant_details.category and uses $sum to calculate the total amount for each category. Second, the $project stage removes the _id field and renames it to "category" for clarity. Third, the $sort stage orders results by total_amount in descending order to show highest volumes first. Fourth, the $limit stage restricts output to the top 6 categories






# Task 5.2

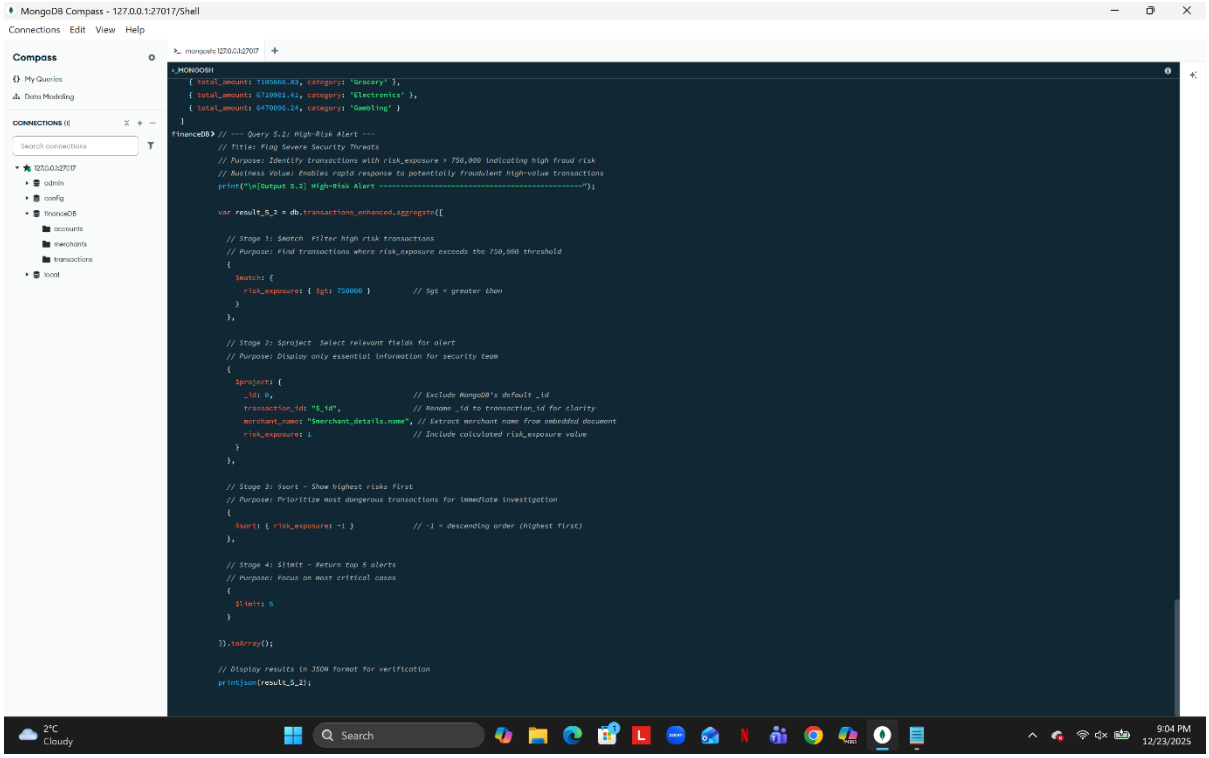
Query Title: Critical Security Threat Detection

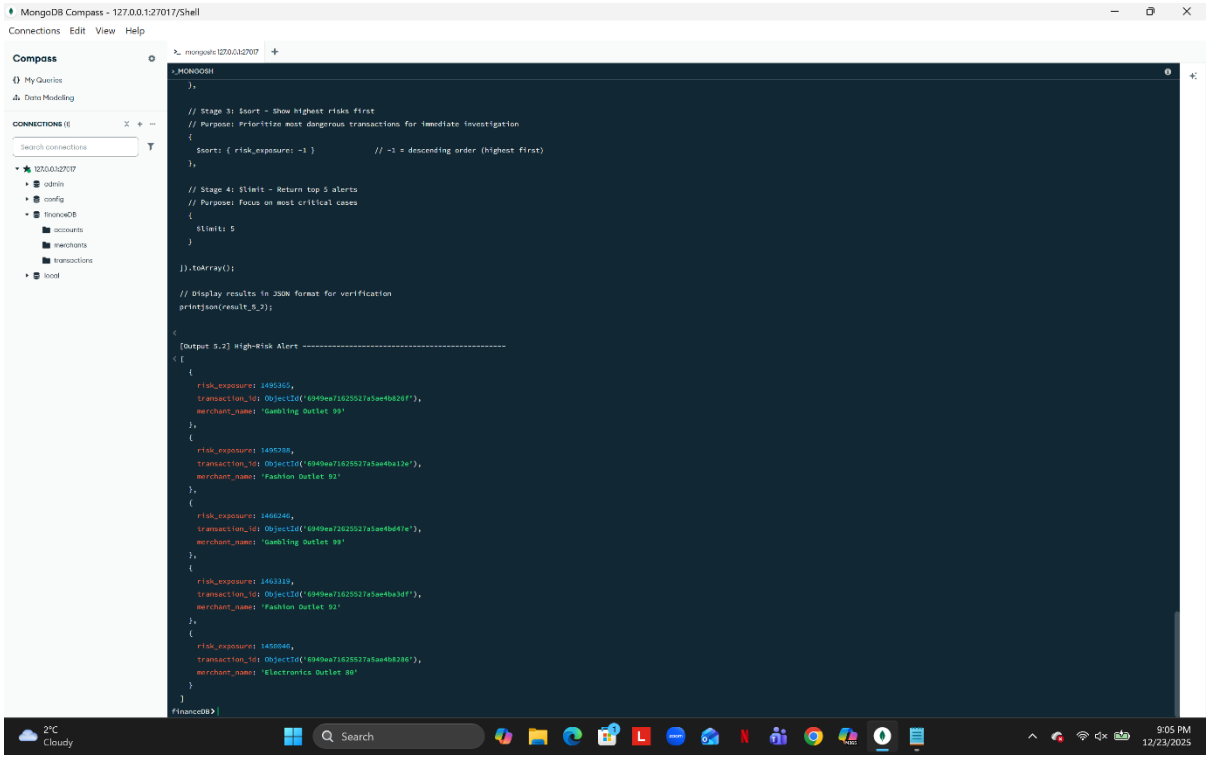
Purpose (Semantic Explanation): This query flags transactions with risk exposure exceeding 750,000 to identify severe security threats requiring immediate investigation.

Process (Technical Explanation):

The query uses a four-stage aggregation pipeline. First, the $match stage filters documents where risk_exposure is greater than 750,000. Second, the $project stage selects specific fields: transaction_id (renamed from _id), merchant_name (from merchant_details.name), and risk_exposure. Third, the $sort stage orders results by risk_exposure in descending order to prioritize highest risks. Fourth, the $limit stage restricts output to the top 5 most critical transactions.

Screenshots of:




# Task 5.3

Query Title: Geographic Spending Pattern Analysis

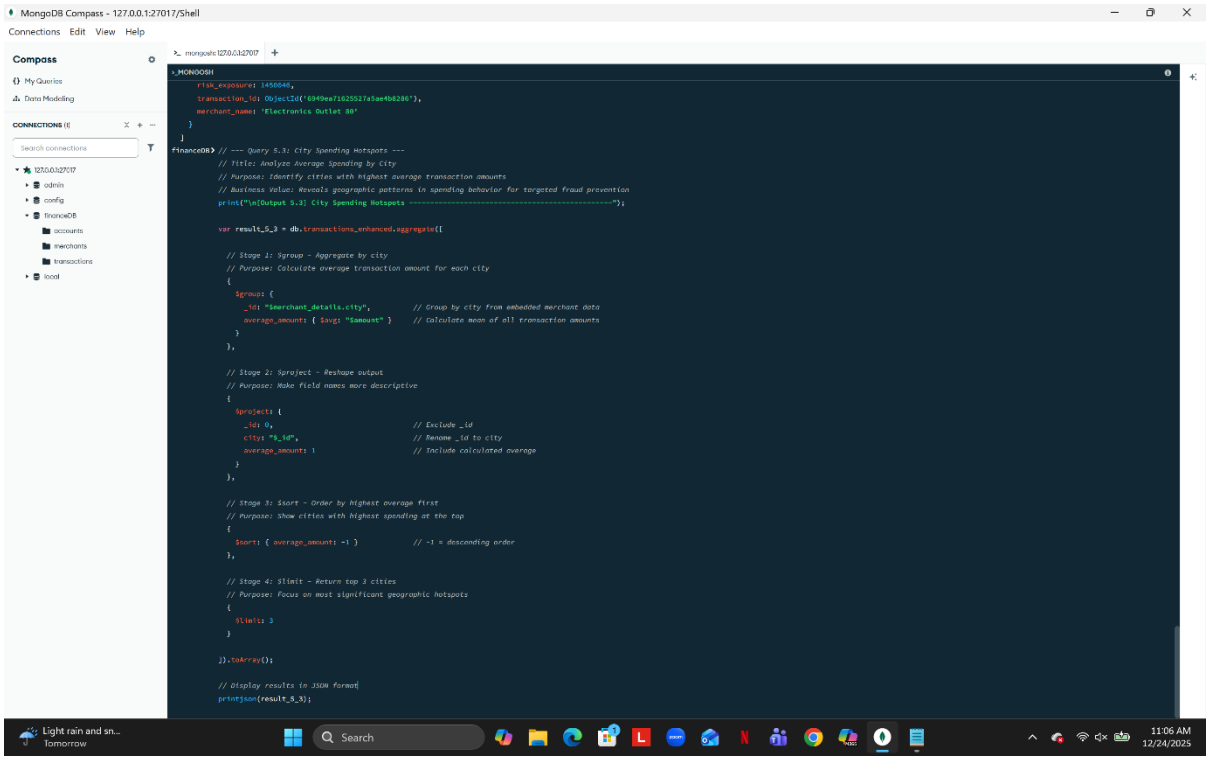
Purpose (Semantic Explanation): This query analyzes average transaction amounts by city to identify geographic regions with the highest spending patterns.

Process (Technical Explanation):

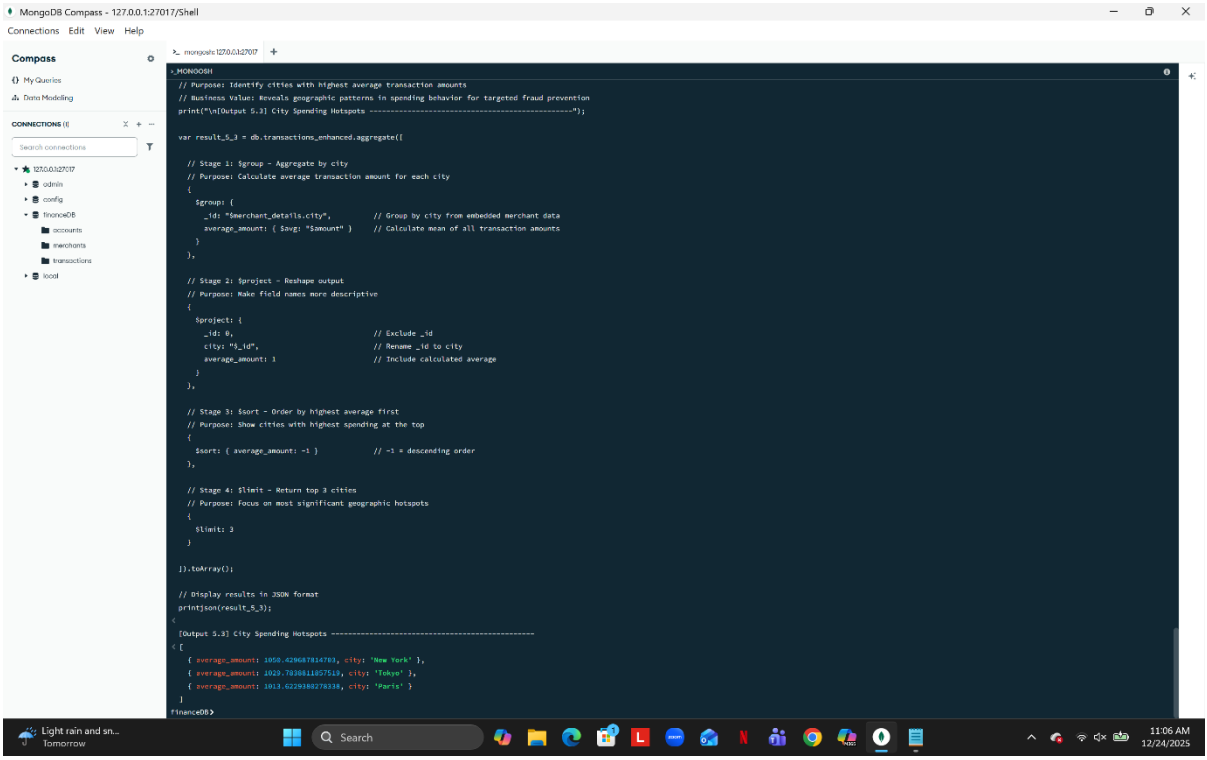
The query uses a four-stage aggregation pipeline. First, the $group stage groups transactions by merchant_details.city and calculates the average amount using $avg. Second, the $project stage removes the _id field and renames it to "city". Third, the $sort stage orders results by average_amount in descending order. Fourth, the $limit stage restricts output to the top 3 cities.

Screenshots of:

b. The query itself.



c. The output of the query




# Task 5.4

Query Title: Most Active Merchant Identification

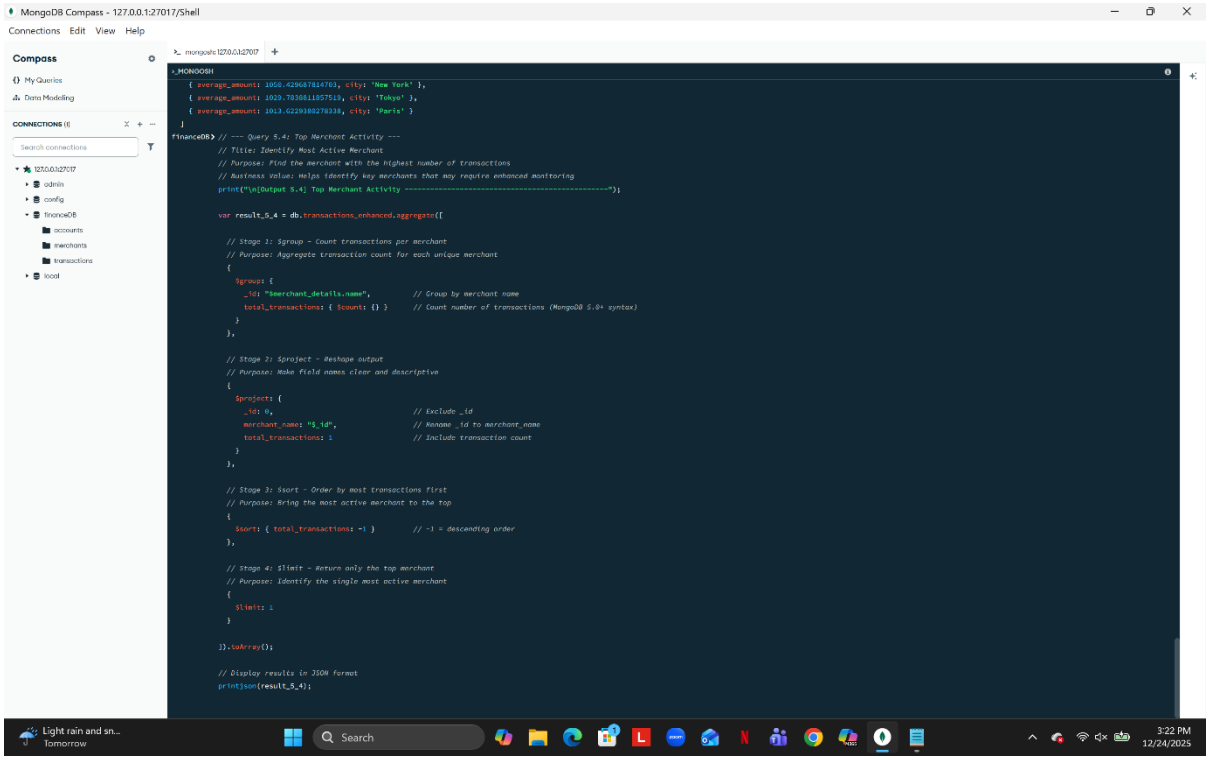
Purpose (Semantic Explanation): This query identifies the merchant with the highest number of transactions to determine which business requires enhanced monitoring.

Process (Technical Explanation):

The query uses a four-stage aggregation pipeline. First, the $group stage groups transactions by merchant_details.name and counts documents using $count. Second, the $project stage removes the _id field and renames it to "merchant_name". Third, the $sort stage orders results by total_transactions in descending order. Fourth, the $limit stage restricts output to exactly 1 document showing the most active merchant.

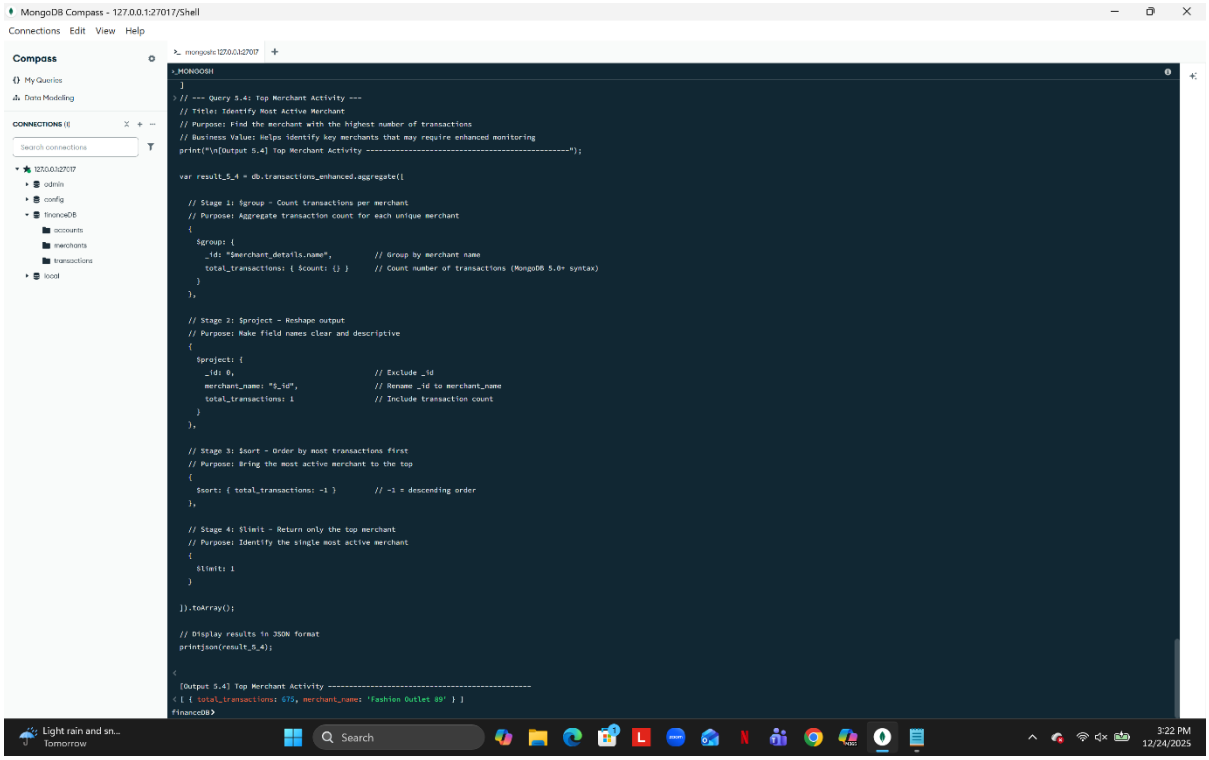
Screenshots of:

1. The query itself.



```
financeDB> // --- Query 5.4: Top Merchant Activity ---
           // Title: Identify Most Active Merchant
           // Purpose: Find the merchant with the highest number of transactions
           // Business Value: Helps identify key merchants that may require enhanced monitoring
           print("\n[Output 5.4] Top Merchant Activity ----------------------------------------");

           var result_5_4 = db.transactions_enhanced.aggregate([

             // Stage 1: $group - Count transactions per merchant
             // Purpose: Aggregate transaction count for each unique merchant
             {
               $group: {
                 _id: "$merchant_details.name",          // Group by merchant name
                 total_transactions: { $count: {} }      // Count number of transactions (MongoDB 5.0+ syntax)
               }
             },

             // Stage 2: $project - Reshape output
             // Purpose: Make field names clear and descriptive
             {
               $project: {
                 _id: 0,                                 // Exclude _id
                 merchant_name: "$_id",                  // Rename _id to merchant_name
                 total_transactions: 1                   // Include transaction count
               }
             },

             // Stage 3: $sort - Order by most transactions first
             // Purpose: Bring the most active merchant to the top
             {
               $sort: { total_transactions: -1 }         // -1 = descending order
             },

             // Stage 4: $limit - Return only the top merchant
             // Purpose: Identify the single most active merchant
             {
               $limit: 1
             }

           ]).toArray();

           // Display results in JSON format
           printjson(result_5_4);
```

a. The output of the query



```
financeDB> // --- Query 5.4: Top Merchant Activity ---
           // Title: Identify Most Active Merchant
           // Purpose: Find the merchant with the highest number of transactions
           // Business Value: Helps identify key merchants that may require enhanced monitoring
           print("\n[Output 5.4] Top Merchant Activity ----------------------------------------");

           var result_5_4 = db.transactions_enhanced.aggregate([

             // Stage 1: $group - Count transactions per merchant
             // Purpose: Aggregate transaction count for each unique merchant
             {
               $group: {
                 _id: "$merchant_details.name",          // Group by merchant name
                 total_transactions: { $count: {} }      // Count number of transactions (MongoDB 5.0+ syntax)
               }
             },

             // Stage 2: $project - Reshape output
             // Purpose: Make field names clear and descriptive
             {
               $project: {
                 _id: 0,                                 // Exclude _id
                 merchant_name: "$_id",                  // Rename _id to merchant_name
                 total_transactions: 1                   // Include transaction count
               }
             },

             // Stage 3: $sort - Order by most transactions first
             // Purpose: Bring the most active merchant to the top
             {
               $sort: { total_transactions: -1 }         // -1 = descending order
             },

             // Stage 4: $limit - Return only the top merchant
             // Purpose: Identify the single most active merchant
             {
               $limit: 1
             }

           ]).toArray();

           // Display results in JSON format
           printjson(result_5_4);

[Output 5.4] Top Merchant Activity ----------------------------------------
[ { total_transactions: 675, merchant_name: 'Fashion Outlet 89' } ]
financeDB>
```

# Conclusions

This assignment involved setting up MongoDB on my system and creating a fraud detection database for FinSecure Banking. I installed MongoDB Community Server and Compass, then created the financeDB database with three collections: transactions, accounts, and merchants. After importing the JSON data files, I corrected data type issues by converting strings to appropriate numeric and date formats. I then created a denormalized collection called transactions_enhanced that embedded merchant information into each transaction and calculated risk exposure values. Finally, I wrote four analytical queries to identify high-volume categories, flag risky transactions, analyze spending by city, and find the most active merchant.

The main challenge I faced was dealing with data types. When I imported the JSON files, data imported as strings, which prevented calculations from working properly. I had to learn how to use MongoDB's type conversion operators like $toInt, $toDouble, and $toDate to fix this. It was different from relational databases where the schema automatically enforces types.

Learning to work with nested documents was also tricky. After joining collections with $lookup and $unwind, I had to remember to access fields like merchant_details.category instead of just category. It took some practice to get the syntax right.

The denormalization concept felt counterintuitive at first since I was taught to avoid data duplication. However, I now understand why MongoDB does this. By embedding merchant data directly into transactions, queries run much faster because the database doesn't need to perform joins. The tradeoff is data redundancy, but for analytical workloads like fraud detection where reads far outnumber writes, this approach makes sense.

Document databases like MongoDB differ significantly from relational databases in their design philosophy. Relational databases normalize data to eliminate redundancy and use joins to connect related information. MongoDB optimizes for query performance by allowing you to store related data together in the same document. This means a transaction document can contain all the merchant information it needs without requiring a separate lookup. While this duplicates some data, it makes queries faster and simpler.

MongoDB's schema flexibility is useful but requires discipline. You can add new fields to documents without altering table structures, but you also need to ensure data quality yourself since there's no automatic schema enforcement. The aggregation pipeline approach is more explicit than SQL, requiring you to specify each transformation step, but it gives you fine control over how data is processed.

For this fraud detection system, MongoDB works well because the team needs to run many different analytical queries quickly. The denormalized structure supports fast queries by category, city, risk level, and other dimensions without complex joins. However, if merchant data changed frequently and needed to update across all transactions, a relational database might be more practical since updates would only affect one table rather than millions of documents.

Theoretical Questions

Question 1: What is the main advantage of denormalization in MongoDB for analytical queries?

Denormalization in MongoDB improves query performance by embedding related data within the same document, eliminating the need for joins. In the fraud detection system, embedding merchant details directly into transaction documents allows queries to retrieve all necessary information in a single read operation. This significantly reduces query execution time compared to relational databases where similar queries would require joining multiple tables. The tradeoff is data redundancy and more complex update operations, but for read-heavy analytical workloads, the performance benefit outweighs these drawbacks.

Question 2: Why is data type correction important in MongoDB when importing JSON files?

Data type correction is essential because MongoDB does not automatically enforce schema or data types during import, often storing numeric and date values as strings. This causes problems with calculations, comparisons, and sorting operations. For example, string values "100" and "20" would sort as 100, 20 alphabetically rather than 20, 100 numerically. In the fraud detection system, keeping amounts and risk scores as strings would prevent proper mathematical operations like calculating risk exposure or summing transaction totals. Converting strings to appropriate types (Int32, Double, Date) using operators like $toInt and $toDouble ensures data can be processed correctly for analytical queries.